OOM Protect

Workstation OOM hardening for systemd Linux
System post-mortem 2026-04-28
2026-04-28

OOM Protect
Project layout
Quick start
Make targets
What `make verify` actually checks
Documentation
Hardware context (for whoever inherits this machine)
Companion alias suggestions
License & credits
- License

OOM Protect

A small, reproducible toolkit that ensures **memory exhaustion never again takes down the whole user session** on this systemd Linux laptop. It was born from the 2026-04-28 incident on host `nezha` where 60+ user processes (Claude Code instances, Android build, MCP servers, IDE, browsers, terminals) were destroyed in a single second when the kernel OOM-killer SIGKILL'd the systemd user-manager itself. See `Crash_Report.md` for the full post-mortem.

The toolkit has three moving parts:

1. **`oom-hardening.sh`** — applies system-wide protection (the umbrella): `systemd-oomd`, cgroup limits on `user-.slice`, sysctls, logind power-key hardening, coredump retention.
2. **`oom-runner.sh`** — wraps individual workloads (the per-app limit): any command can be launched in its own bounded scope/service so a single runaway dies in isolation.
3. **`oom-watch/`** — the monitoring daemon. A small Go service that uses `atop` to sample system state every 10 s, evaluates a threshold ladder (NOTICE → WARN → CRITICAL), and writes a detailed Markdown forensic report **just before** thresholds breach so you have evidence at incident time, not after the fact. Deploy with `sudo make oomwatch-deploy`. See `manuals/oom-watch-deployment-guide.md` and `manuals/oom-watch-runbook.md`.

Use all three together. The first sets the umbrella; the second sets the per-leaf bounds inside it; the third writes forensic evidence the moment either is being stressed.

Project layout

```
~/Downloads/
├── README.md                          ← this file
├── Makefile                           ← convenience targets (make help)
│
├── Crash_Report.md / .html / .pdf     ← post-mortem of 2026-04-28 incident
│
├── oom-hardening.sh                   ← applies system-wide OOM hardening
├── oom-runner.sh                      ← runs ANY command in a bounded cgroup
├── build-docs.sh                      ← rebuilds HTML + PDF from .md
├── verify.sh                          ← health check (returns non-zero if not green)
│
├── manuals/
│   ├── oom-hardening-manual.md / .html / .pdf
│   └── oom-runner-manual.md    / .html / .pdf
│
├── assets/
│   └── style.css                      ← shared CSS (screen + print)
│
└── system-config/                     ← staging tree (informational)
```

Quick start

```
# 0. Read the post-mortem first
xdg-open ~/Downloads/Crash_Report.pdf

# 1. Preview hardening, change nothing
make -C ~/Downloads dry-run

# 2. Apply hardening (sudo)
make -C ~/Downloads install

# 3. Verify everything is green
make -C ~/Downloads verify

# 4. Use oom-runner for daily workloads
~/Downloads/oom-runner.sh --preset claude  -- claude
~/Downloads/oom-runner.sh --preset mcp     -n upstash -- npm exec @upstash/context7-
        mcp@latest
~/Downloads/oom-runner.sh --preset build   -- bash -c 'cd ~/proj && m -j8'

# 5. (Optional) Stress-test the umbrella
make -C ~/Downloads verify-stress
```

Add to your shell rc to make `oom` a one-letter command:

```
# ~/.bashrc or ~/.zshrc
alias oom='~/Downloads/oom-runner.sh'
alias claude='~/Downloads/oom-runner.sh --preset claude -- claude'
```

Make targets

Run `make help` for the live list. Highlights:

Target	Effect	Sudo?
`make help`	Print all targets (default)	no
`make dry-run`	Preview hardening; change nothing	no
`make install`	Apply hardening	yes
`make verify`	Health check; non-zero on warn/fail	no
`make verify-stress`	Verify + 16 G stress test	no
`make uninstall`	Remove drop-ins this toolkit installed	yes
`make rollback BACKUP=DIR`	Restore /etc from a backup directory	yes
`make docs`	Rebuild all HTML + PDF	no
`make docs-clean`	Delete generated HTML + PDF	no
`make presets`	List oom-runner presets	no
`make list`	List active oom-runner units	no
`make status UNIT=x`	Show one unit’s live status	no
`make logs UNIT=x`	Tail one unit’s journal (`-f`)	no
`make kill UNIT=x`	Stop one unit	no
`make clean-units`	Stop ALL oom-runner units	no
`make test`	Functional smoke test	no
`make check`	Lint shell scripts (`bash -n` + shellcheck if available)	no
`make package`	Bundle into `oom-toolkit-<ts>.tar.gz`	no
`make all`	docs + verify (typical CI)	no
`make clean`	Remove generated files	no

What `make verify` actually checks

Check	Source of truth
All toolkit files present	filesystem
Kernel ≥ 5.15	`uname -r`
systemd ≥ 248	`systemctl --version`
cgroup v2 unified hierarchy	`mount`
PSI available	`/proc/pressure/memory`
`systemd-oomd.service` active	`systemctl is-active`
oomd is managing `user.slice`	`oomctl` output
`user-<uid>.slice` has memory limits	`systemctl show`
`logind HandlePowerKey=ignore`	`busctl get-property`
`vm.swappiness=10`	`sysctl -n`
`vm.min_free_kbytes=262144`	`sysctl -n`
`vm.overcommit_memory=0`	`sysctl -n`
`vm.dirty_ratio=15`	`sysctl -n`
`vm.dirty_background_ratio=5`	`sysctl -n`
`vm.vfs_cache_pressure=50`	`sysctl -n`
Coredump retention drop-in present	filesystem
Swap configured	`/proc/meminfo`
`oom-runner.sh` can launch a 256M scope	live exec
Cgroup actually kills processes that exceed limit	live OOM test
(with `--stress`) `stress-ng --vm 4 --vm-bytes 16G` is killed	live

Exit code: `0` = all green, `1` = warnings only, `2` = at least one failure.

`verify.sh --json` produces machine-readable output for CI/monitoring.

Documentation

Each component has a full manual:

- `Crash_Report.md` — what happened on 2026-04-28, why, and what to do.
- `manuals/oom-hardening-manual.md` — every option, drop-in, tuning recipe.
- `manuals/oom-runner-manual.md` — every option, preset, scenario.
- `manuals/oom-watch-manual.md` — feature reference for the monitoring daemon: every config key, severity ladder, report anatomy.
- `manuals/oom-watch-deployment-guide.md` — install and verify the daemon end-to-end. Includes ALT Linux specifics, a real session walk-through, and the diagnostic-bundler procedure.
- `manuals/oom-watch-runbook.md` — incident-response playbook. Every issue we have hit in production or during deployment is documented with one-command diagnosis and exact remediation. Read once; bookmark for 3 a.m.
- `reports/oom-watch-architecture.md` — design decisions for the daemon.

All three are also rendered as `.html` (standalone, embedded CSS) and `.pdf` (via weasyprint, A4 paged media). Re-render any time after editing:

```
make docs
```

Hardware context (for whoever inherits this machine)

Item	Value
Vendor / model	Clevo NS50MU
CPU	Intel Core i7-1165G7 (Tiger Lake-UP3, 4C / 8T)
Installed RAM	64 GB (`MemTotal: 65,625,040 kB`)
Max RAM the CPU can address	**64 GB** (Intel ARK spec)
Swap	16 GiB on NVMe
Kernel	6.12.61-6.12-alt1
Distro	ALT Linux

Going to 128 GB on this laptop is **not possible** — the i7-1165G7 IMC caps at 64 GB regardless of physical SODIMM size. See `Crash_Report.md` §6.

Companion alias suggestions

```
# ~/.bashrc / ~/.zshrc
alias oom='~/Downloads/oom-runner.sh'
alias oom-list='~/Downloads/oom-runner.sh list'
alias oom-clean='~/Downloads/oom-runner.sh clean'
alias oom-verify='~/Downloads/verify.sh'

# Wrap your most common commands
alias claude='~/Downloads/oom-runner.sh --preset claude -- claude'
alias chromium='~/Downloads/oom-runner.sh --preset browser -- chromium'
```

License & credits

Internal incident-response artefact. Use freely on this host. No warranty.

Built 2026-04-28 in response to the user-manager OOM-kill incident at 18:36:35 MSK on `nezha`. Total downtime: 7 minutes. Total work lost: significant. Probability of recurrence after `make install`: very low.

License

Copyright (c) Milos Vasic. Licensed under the Apache License, Version 2.0 – see `LICENSE`.